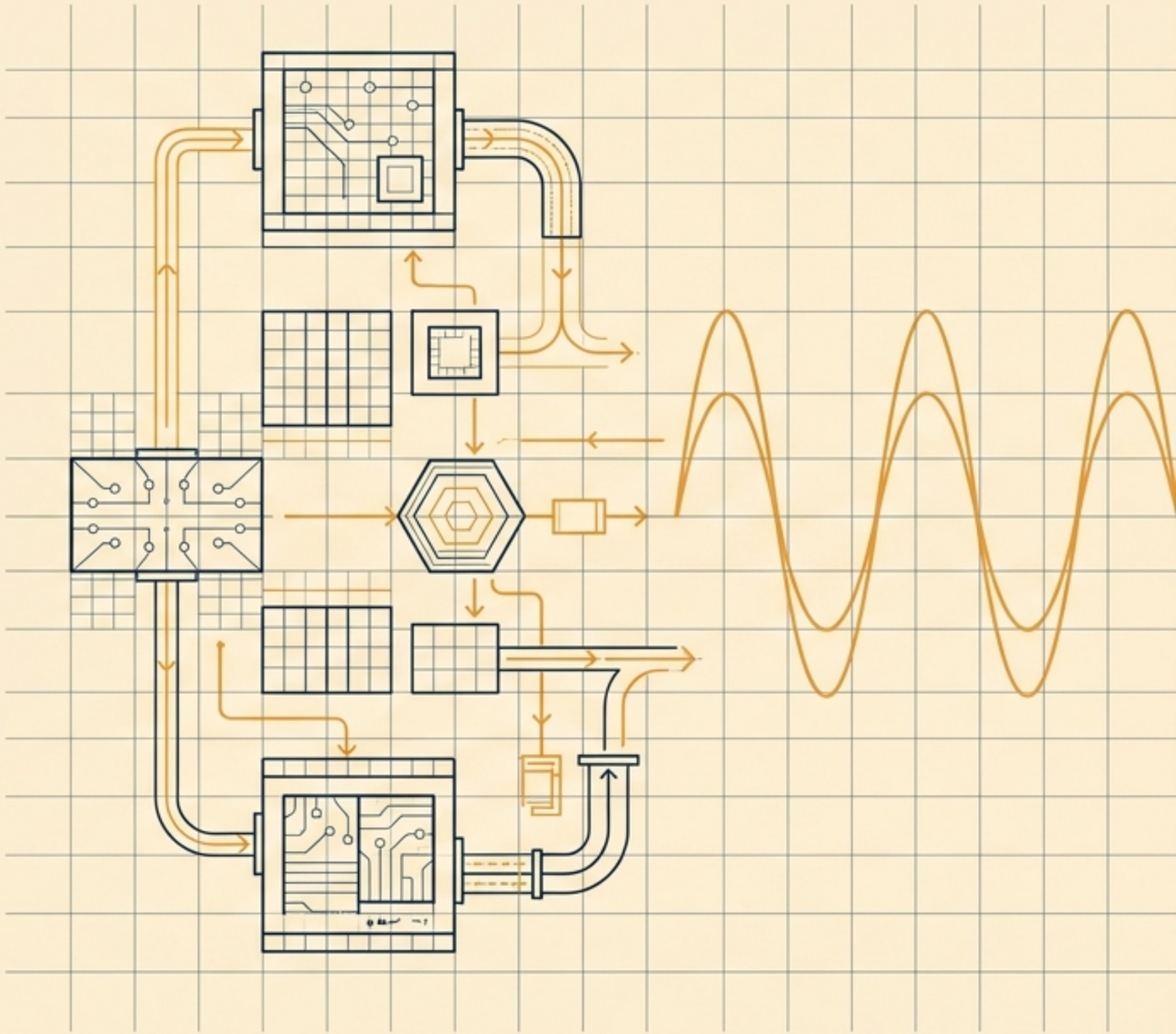
GCS: The Software Engine

Architecting the **Telemetry Backend** for **Sounding Rockets** and **Model Satellites**



Part 1: Hardware Bridging, De-commutation, and Zero-Latency Transport

NotebookLM

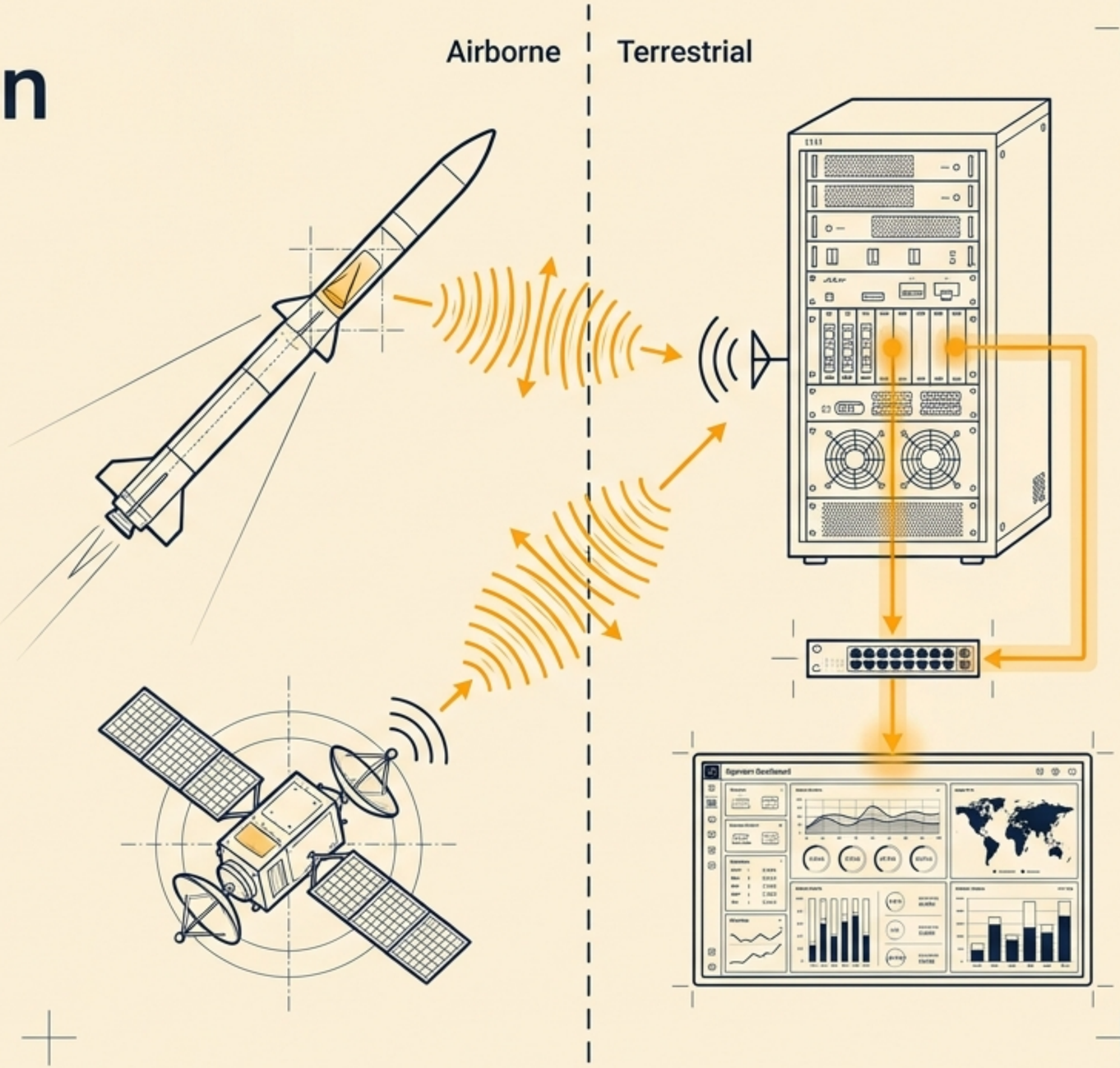
Telemetry Acquisition & State Monitoring Infrastructure

The Digital Nervous System

A Ground Control Station (GCS) is the terrestrial nerve center of an aerospace mission. Think of it as a hospital bedside monitor for a patient moving at hundreds of meters per second. Once a vehicle leaves the pad, there is no physical access. Every measurement—from barometric altitude to gyroscopic rotation—must be wirelessly captured, translated, and presented to operators in fractions of a second to enable real-time diagnostic decisions.

The 50% / 40% Split

Designed to accommodate the high-G trajectories of Sounding Rockets (50%) and the sustained orbital data streams of Model Satellites/CanSats (40%), requiring extreme resilience to packet loss and signal shadowing.



NotebookLM

Layered De-commutation & Routing Pipeline

The Universal Translator. Data arriving from the sky is fast, dense, and messy. The software engine acts as a universal translator, catching raw, scrambled radio signals and assembling them into perfectly structured sentences before the next batch arrives. This pipeline operates in three distinct, hyper-specialized phases.

Layer 1: Hardware Backend

Bridge & Parse

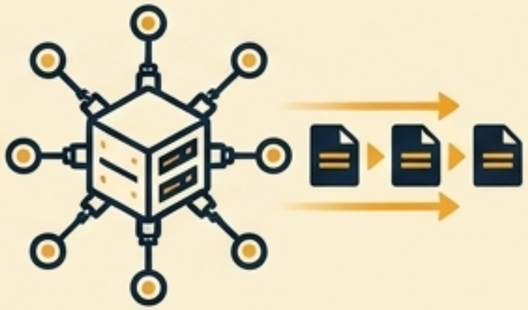


Ingests raw bytes via USB/Serial from the LoRa receiver. Validates physically plausible ranges and structures data.

Raw Byte Stream → Parsed JSON

Layer 2: Transport Protocol

Stream



Maintains a persistent pipeline to the frontend interface. Avoids heavy request overhead.

Parsed JSON → Network Push

Layer 3: Visualization

Render



Distributes telemetry into isolated sub-systems (Graphs, 3D Canvas, Map).

Network Push → DOM Update

NotebookLM

Concurrency & I/O Benchmark: Backend Selection

Choosing the Right Engine

Building a backend requires matching the software to the physics of the data. Do you need a massive freight train, a highly-regulated courier, or a fleet of rapid-delivery drones? For 1 to 50Hz rocketry telemetry, we prioritize asynchronous I/O and serial hardware compatibility over heavy computational lifting.

Dimension	Node.js (V8)	Python + FastAPI	Go (Golang)
Architecture	Event-driven, Non-blocking I/O	Synchronous core, Async via ASGI	Goroutines, high-concurrency
Data Validation	Custom typed schemas	Pydantic (Strict, robust parsing)	Struct tagging
Serial Port Ecosystem	Excellent (Native cross-platform) ✓	Good (PySerial)	Moderate (Requires CGO bindings)
1-50Hz Suitability	**Optimal**. Handles concurrent serial ingestion & WebSocket streaming flawlessly in a single language stack.	**Viable**. Superb validation, but GIL/threading overhead complicates hard real-time serial+web async loops. ⚠	**Overkill**. Unmatched concurrency, but introduces unnecessary compilation complexity for sub-100Hz hardware bridging. ✗

NotebookLM

Serial-to-Network Bridging & Payload De-commutation

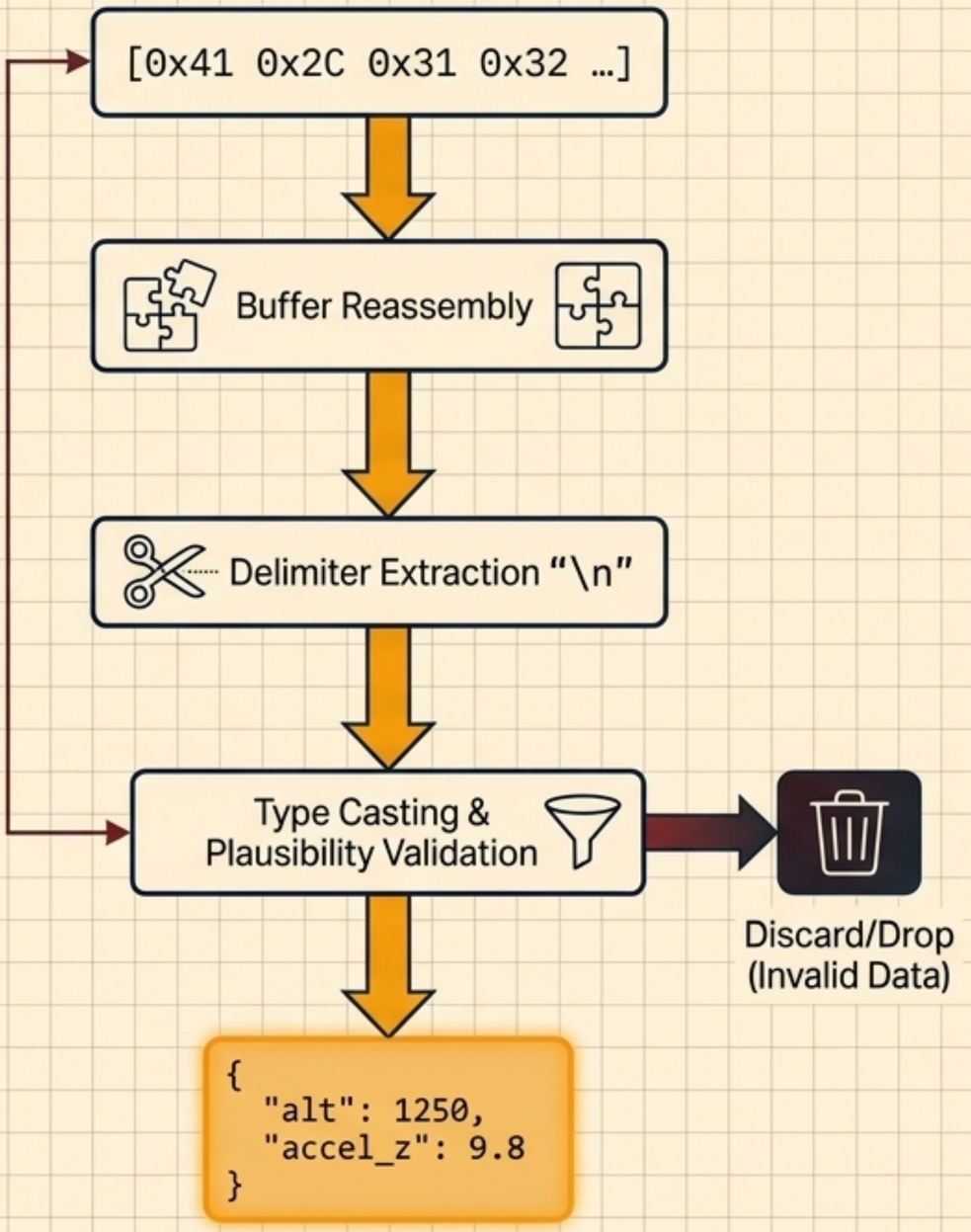
The Intake Valve

Radio telemetry is not a clean file transfer; it is a chaotic, high-pressure river of bytes. The software bridge acts as an intake valve and filtration system. It captures fragmented pieces, stitches them together, discards corrupted droplets, and packages the clean data into neat digital boxes (JSON) for the dashboard.

Fragmentation Handling: Serial data rarely arrives whole. The buffer accumulates bytes until a termination character (\n) triggers extraction.

Plausibility Validation: LoRa packet loss causes dropped fields. A single corrupted byte creates wildly inaccurate scaling (NaN). Invalid lengths or or physics-defying values are silently discarded.

To JSON: Clean ASCII/Binary is strictly type-cast from text strings into numerical JavaScript Objects, ready for network routing.

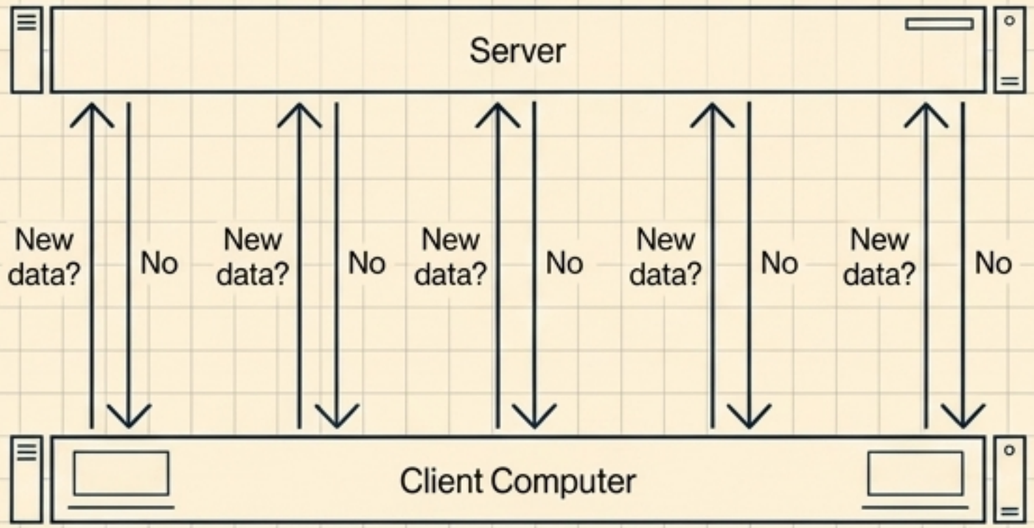


NotebookLM

Persistent Bidirectional Transport (Sub-10ms Latency)

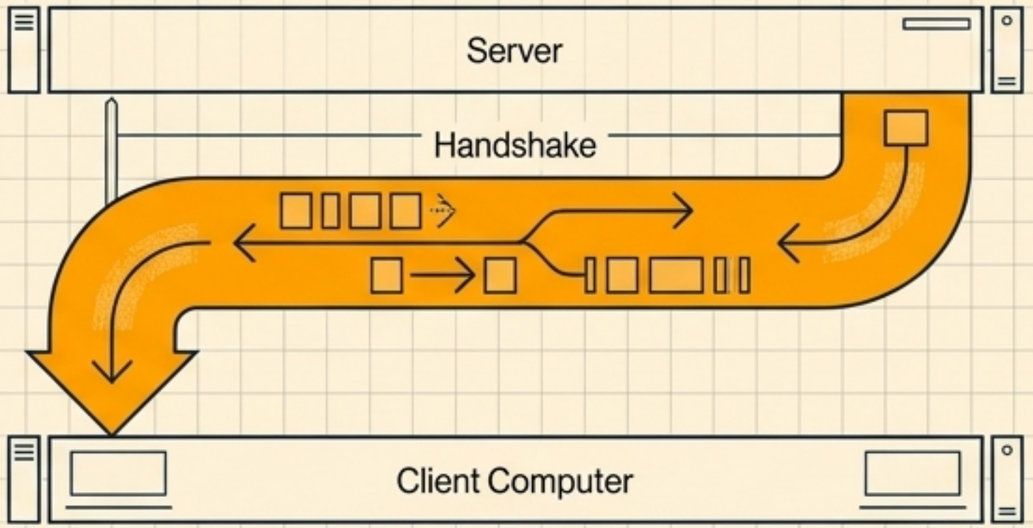
The Open Pipe. To get data from the computer to the screen instantly, traditional web methods are too slow. Standard HTTP is like hanging up and dialing a phone number 50 times a second to ask for an update. WebSockets establish a single, open pipe. The connection stays live, allowing the server to push new telemetry to the screen the exact millisecond it arrives.

HTTP Polling (The Bottleneck)



Mechanism: Request-Response cycle. High bandwidth overhead from headers. Unsuitable for 10–50Hz telemetry; causes UI stutter and CPU drain.

WebSocket Protocol (The Standard)



Mechanism: Single TCP handshake creates a full-duplex connection. End-to-end latency under 10ms. Data is pushed entirely unprompted for hard-real-time DOM updates.

NotebookLM